Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación del Entregable: Documento de Arquitectura de Software (SAD), Modelo C4 y Patrones de Diseño Modular (EDT 2.2)

2. INTRODUCCIÓN

La arquitectura de software del sistema **SIGE** se concibe bajo principios de alta cohesión, bajo acoplamiento, escalabilidad, máxima velocidad de respuesta e independencia modular. Se implementa una **Arquitectura Modular en Capas basada en Node.js + Express (v5)** y **PostgreSQL (18)** en el backend, articulada con una interfaz **Single Page Application (SPA) en React + Vite** potenciada por **Tailwind CSS**, animaciones fluidas por GPU mediante CSS Keyframes y paleta institucional `#00345E` (Azul UTA/FISEI). Esta base técnica se complementa con el **Modelo C4** para la visualización estructural del sistema y especificaciones rigurosas de seguridad basadas en **Bcrypt.js**, **JSON Web Tokens (JWT)** y cabeceras de protección HTTP con **Helmet**.

3. OBJETIVO

3.1. Objetivo General

Diseñar y formalizar la arquitectura modular de software del sistema SIGE mediante Node.js + Express (v5), PostgreSQL 18, React + Vite y el Modelo C4, asegurando mantenibilidad, testabilidad, rendimiento ultra-rápido y confidencialidad en el flujo documental institucional.

3.2. Objetivos Específicos

1. Estructurar la arquitectura modular en capas del backend Node.js + Express (v5) (Configuración, Base de Datos, Middlewares, Módulos de Dominio y Utilidades).
2. Modelar el Diagrama de Contenedores C4 que articula Frontend React + Vite, Backend Express v5, PostgreSQL 18, MinIO S3, LanguageTool en Docker local y Google Gemini API.
3. Especificar la estructura monorepo canónica del proyecto (`SIGE/`) con backend modular y frontend SPA.
4. Definir los patrones de diseño aplicados (Modular Monolith, Controller-Service-Repository, Validation Gates en middlewares, Connection Pooling e Interceptores Axios).
5. Formalizar los mecanismos de seguridad criptográfica con Bcrypt.js, firmas JWT HMAC-SHA256 y cabeceras HTTP con Helmet.

4. ALCANCE

4.1. Límites Arquitecturales (In-Scope)

- Estructura monorepo canónica `SIGE/` conteniendo `backend/` (Node.js + Express v5) y `frontend/` (React + Vite).
- Capa de persistencia relacional ACID en PostgreSQL 18 gestionada mediante connection pool `pg`, `schema.sql`, `seed.sql` y script de inicialización `init.js`.
- Comunicación reactiva entre el cliente SPA y la API REST mediante Axios con interceptores de auto-renovación de tokens.
- Protección integral del perímetro HTTP con Helmet, rate limiting y CORS institucional.

4.2. Exclusiones (Out-of-Scope)

- Despliegue en clústeres Kubernetes multi-región o balanceadores de carga globales de pago.

5. DEFINICIONES

- **Arquitectura Modular:** Enfoque arquitectónico donde la lógica de negocio se divide en módulos independientes y autocontenidos (`auth/`, `users/`, `reports/`) con controladores, servicios y repositorios propios.
- **Modelo C4:** Marco de trabajo para la diagramación de arquitectura en cuatro niveles jerárquicos (Contexto, Contenedores, Componentes y Código).
- **Express (v5):** Framework web minimalista y robusto para Node.js con soporte nativo de promesas en enrutamiento y manejo mejorado de errores asíncronos.
- **JWT (*JSON Web Token*):** Estándar abierto (RFC 7519) para transmitir identidades firmadas criptográficamente.
- **Helmet:** Middleware de seguridad para Express que configura cabeceras HTTP de protección (X-Content-Type-Options, X-Frame-Options, HSTS, CSP).

6. RESPONSABILIDADES

Integrante	Rol en el Entregable	Responsabilidad Específica
Steeven Toala	Líder / Arquitecto	Diseño de la arquitectura modular, diagramación C4, especificación de patrones y seguridad JWT/Helmet.
Nixon Hurtado	Backend / DBA	Estructuración del pool PostgreSQL 18, scripts DDL `schema.sql` y `init.js`.
Ing. Paulo Torres	Docente Tutor	Revisión y aprobación del documento de arquitectura de software.

Cuadro 1: Matriz de Responsabilidades del Entregable EDT 2.2.

7. NORMATIVA LEGAL Y TÉCNICA

- **IEEE Std 1471-2000 / ISO/IEC 42010:2011:** Systems and software engineering – Architecture description.
- **OpenAPI Specification 3.0 (OAS):** Estándar para documentación de contratos REST.
- **RFC 7519:** JSON Web Token (JWT) Standard.
- **OWASP Top 10 API Security:** Buenas prácticas de mitigación de vulnerabilidades web y APIs.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Capas de la Arquitectura Modular Node.js + Express (v5)

Para asegurar alta mantenibilidad, desacoplamiento y testabilidad rigurosa, el backend de **SIGE** se organiza en 6 niveles jerárquicos modulares:

1. **Capa de Rutas y Aplicación (`app.js`):** Configuración centralizada de Express (v5), cabeceras Helmet, políticas CORS, deserialización JSON y montaje de enrutadores.
2. **Capa de Seguridad y Middlewares (`middlewares/`):** Filtros de autenticación Bearer JWT, limitadores de tasa (*rate limiters*), compuertas Validation Gates y manejador global de excepciones.

3. **Controladores Modulares (`modules/*/controller`):** Orquestadores de peticiones HTTP, extracción de parámetros y delegación a los servicios.
4. **Servicios de Dominio (`modules/*/service`):** Implementación estricta de las reglas de negocio, hashing criptográfico y generación de respuestas.
5. **Repositorios de Datos (`modules/*/repository`):** Consultas SQL parametrizadas directas para evitar inyecciones SQL y control de transacciones ACID.
6. **Persistencia Relacional ACID (`database/`):** Motor PostgreSQL 18 gestionado mediante un Connection Pool optimizado con el controlador `pg`.

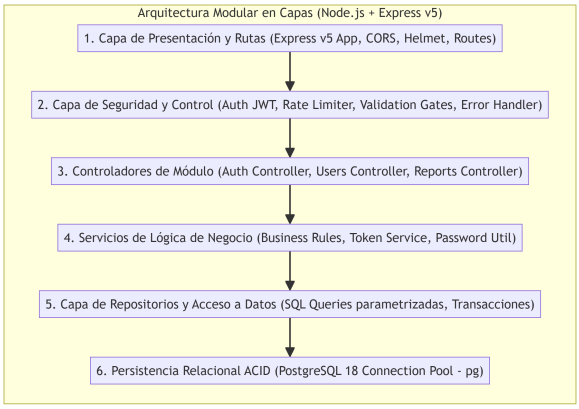


Figura 1: Arquitectura Modular en Capas para Node.js + Express (v5).

8.2. Diagrama de Contenedores C4 de la Solución

El modelo C4 articula la interacción entre el cliente SPA y los servicios locales y externos:

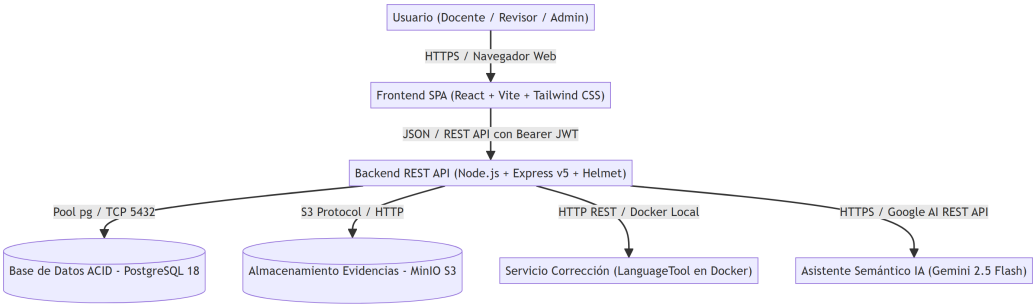


Figura 2: Diagrama de Contenedores del Modelo C4 para el Ecosistema SIGE.

- **Frontend SPA:** React + Vite con Tailwind CSS y CSS Keyframes institucionales (`#00345E`).
- **Backend REST API:** Node.js + Express (v5) (puerto 5000) protegido por Helmet y Bearer JWT.
- **Base de Datos Relacional:** PostgreSQL 18 (puerto 5432) con almacenamiento ACID y pool de conexiones.
- **Almacenamiento de Evidencias:** MinIO S3 (puertos 9000 API y 9001 Consola Web).
- **Servicio de Corrección Sintáctica:** LanguageTool local en contenedor Docker (puerto 8010).
- **Asistente Semántico IA:** Inferencia delegada a la API de Google Gemini mediante peticiones HTTPS seguras.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Estructura Monorepo Canónica del Proyecto (SIGE/)

```
SIGE/
|-- .gitignore                  # Reglas de exclusion para Git (node_modules, .env, dist)
|-- package.json                # Configuracion raiz
|-- backend/
|   |-- src/
|   |   |-- config/             # Conexion PostgreSQL (pool) y entorno
|   |   |-- database/           # schema.sql, seed.sql y script init.js
|   |   |-- middlewares/        # auth.middleware, rateLimiter, errorHandler
|   |   |-- modules/
|   |   |   |-- auth/           # Controller, Service, Repository, Validation, Routes
|   |   |   `-- users/          # Controller, Service, Repository, Validation, Routes
|   |   |-- utils/              # password.util (bcrypt), token.util (JWT), response.util
|   |   |-- app.js              # Express app, CORS, Helmet, parseo JSON, rutas montadas
|   |   `-- server.js           # Inicio del servidor HTTP y escucha de conexiones
|   |-- .env                    # Variables locales (PORT, DB_*, JWT_SECRET)
|   |-- .env.example            # Plantilla de variables de entorno
|   `-- package.json            # Dependencias backend (express 5, pg, bcryptjs, jwt, helmet)
`-- frontend/
    |-- public/                 # Favicon y assets estaticos institucionales (logo-uta, logo-fisei)
    |-- src/
    |   |-- api/                # Cliente Axios con interceptor (JWT en header, auto-renovacion)
    |   |-- components/common/  # UI reutilizable: Button, Input, Modal, Card, Spinner
    |   |-- context/            # AuthContext (login, logout, estado del usuario)
    |   |-- pages/
    |   |   |-- LoginPage.jsx   # Pantalla de acceso institucional (#00345E, CSS Keyframes)
    |   |   `-- DashboardPage.jsx # Panel principal segun rol del usuario
    |   |-- App.jsx             # Rutas protegidas (react-router-dom)
    |   |-- main.jsx            # Punto de entrada React
    |   `-- index.css           # Directivas Tailwind, animaciones GPU y variables institucionales
    |-- .env                    # Variables del frontend (VITE_API_BASE_URL)
    |-- .env.example
    |-- tailwind.config.js      # Configuracion Tailwind (#00345E azul institucional UTA/FISEI)
    |-- vite.config.js          # Configuracion Vite y servidor de desarrollo
    `-- package.json            # Dependencias frontend (react, react-dom, axios, lucide-react)
```

9.2. Patrones de Diseño Implementados

1. **Modular Monolith & Layered Architecture:** Cada dominio (`auth`, `users`, `reports`) agrupa sus rutas, validaciones, controladores, servicios y repositorios de forma desacoplada y mantenible.

2. **Controller-Service-Repository:** Desacopla la gestión del protocolo HTTP (controlador) de las reglas de

negocio (servicio) y el acceso a datos SQL puro parametrizado (repositorio).

3. **Validation Gates en Middlewares:** Validación en cascada donde middlewares especializados rechazan peticiones con esquemas malformados o reglas de negocio incumplidas antes de ejecutar la lógica de aplicación.
4. **Connection Pooling con `pg`:** Reutilización eficiente de conexiones abiertas hacia PostgreSQL 18 para maximizar el throughput y minimizar latencias en transacciones ACID.
5. **Interceptor Pattern en Axios:** Interceptores de request para inyectar automáticamente el header `Authorization: Bearer <token>` e interceptores de response para gestionar la auto-renovación silenciosa con refresh token.
6. **Defense in Depth con Helmet:** Aplicación de 11 middlewares de seguridad en Express v5 para mitigar ataques XSS, Clickjacking, MIME sniffing y degradación SSL.

9.3. Especificación de Seguridad y Token JWT

- **Hashing de Contraseñas:** Bcrypt.js con factor de costo 12 para resistencia contra ataques de fuerza bruta.
- **Algoritmo de Firma Criptográfica:** HMAC-SHA256 con clave secreta de 512 bits inyectada por variable de entorno (`JWT_SECRET`).
- **Tiempo de Expiración:** Access Token de 15 a 30 minutos y Refresh Token seguro con ciclo de vida de 7 días gestionado en la tabla `auth_tokens` de PostgreSQL 18.
- **Claims Oficiales en el Token:**
  - `sub (UserId)`: Identificador único universal del usuario (UUID).
  - `email`: Correo electrónico institucional (`@uta.edu.ec`).
  - `fullName`: Nombre completo del docente o directivo.
  - `role`: Rol RBAC (`DocenteElaborador`, `CoordArea`, `CoordCarrera`, `Admin`).
  - `facultadId` y `carreraId`: Pertenencia institucional a la FISEI e Ingeniería de Software.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Requisitos / UI-UX		31/08/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		31/08/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Diseño de arquitectura modular en capas Node.js + Express (v5), PostgreSQL 18, React + Vite, modelo C4 y especificacion JWT/Helmet	Steeven Toala	**Pendiente de Revisión**

12. ANEXOS

Anexo 1: Inicialización del Servidor HTTP y Middlewares en Express (v5)

Configuración centralizada en `app.js` y `server.js` que demuestra el montaje de Helmet, CORS, parseo JSON, pool de PostgreSQL 18 y el enrutador modular para autenticación y usuarios.